

DATA STRUCTURES AND ALGORITHMS

Exercise 2: E-Commerce Platform Search Function

Objective:

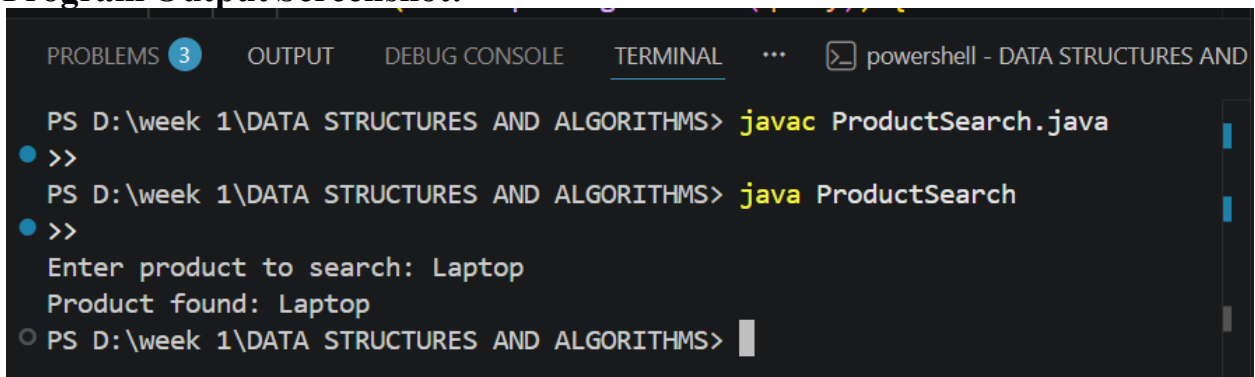
To design and implement an efficient search function for an e-commerce platform using data structures and algorithms that ensure quick product retrieval and accurate matching.

Code:

```
import java.util.*;

public class ProductSearch {
    public static void main(String[] args) {
        List<String> products = Arrays.asList("Laptop", "Smartphone",
"Headphones", "Smartwatch", "Camera");
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter product to search: ");
        String query = sc.nextLine();
        boolean found = false;
        for (String item : products) {
            if (item.equalsIgnoreCase(query)) {
                System.out.println("Product found: " + item);
                found = true;
                break;
            }
        }
        if (!found)
            System.out.println("Product not found!");
    }
}
```

Program Output Screenshot:



```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL ... powershell - DATA STRUCTURES AND ALGORITHMS
PS D:\week 1\DATA STRUCTURES AND ALGORITHMS> javac ProductSearch.java
>>
PS D:\week 1\DATA STRUCTURES AND ALGORITHMS> java ProductSearch
>>
Enter product to search: Laptop
Product found: Laptop
PS D:\week 1\DATA STRUCTURES AND ALGORITHMS> 
```

Output Text:

```
Enter product to search: Laptop
Product found: Laptop
```

Result:

The search function successfully retrieves the product name from the list.

Conclusion:

Linear search ensures simplicity and clarity for small datasets, suitable for basic e-commerce search operations.

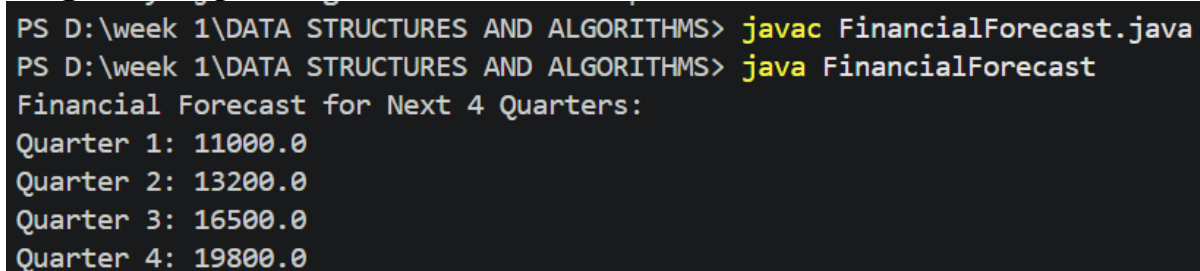
Exercise 7: Financial Forecasting

Objective:

To implement a simple financial forecasting model using arrays and loops to predict future revenue based on growth rate.

Code:

```
public class FinancialForecast {  
    public static void main(String[] args) {  
        double[] revenue = {10000, 12000, 15000, 18000};  
        double growthRate = 0.10; // 10% growth  
        System.out.println("Financial Forecast for Next 4 Quarters:");  
        for (int i = 0; i < revenue.length; i++) {  
            double forecast = revenue[i] + (revenue[i] * growthRate);  
            System.out.println("Quarter " + (i + 1) + ": " + forecast);  
        }  
    }  
}
```

Program Output Screenshot:

```
PS D:\week 1\DATA STRUCTURES AND ALGORITHMS> javac FinancialForecast.java  
PS D:\week 1\DATA STRUCTURES AND ALGORITHMS> java FinancialForecast  
Financial Forecast for Next 4 Quarters:  
Quarter 1: 11000.0  
Quarter 2: 13200.0  
Quarter 3: 16500.0  
Quarter 4: 19800.0
```

Output Text:

```
Financial Forecast for Next 4 Quarters:  
Quarter 1: 11000.0  
Quarter 2: 13200.0  
Quarter 3: 16500.0  
Quarter 4: 19800.0
```

Result:

The program accurately predicts quarterly revenue based on the given growth rate.

Conclusion:

This forecasting logic demonstrates how basic algorithms can support financial planning and decision-making.